



수상작 리뷰

YB 1조 김재경, 유재은, 이우림, 전예원



목차

1. 제주 특산물 가격 예측 AI 경진대회_전예원
2. 기업 성공 확률 해커톤_이우림
3. 주식 종료 가격 예측_유재은
4. 유전체 정보 품종 분류 AI 경진대회_김재경

제주 특산물 가격 예측 AI 경진대회

1. 주제: 제주도 특산물의 가격을 예측하는 AI 모델 개발 및 인사이트 발굴
2. 데이터

train.csv

2019년 01월 01일부터 2023년 03월 03일까지의
유통된 품목의 가격 데이터

- item: 품목 코드
TG : 감귤, BC : 브로콜리, RD : 무, CR : 당근, CB : 양배추
- corporation : 유통 법인 코드
법인 A부터 F
- location : 지역 코드
J : 제주도 제주시, S : 제주도 서귀포시
- supply(kg) : 유통된 물량
- price(원/kg) : 유통된 품목들의 kg 마다의 가격

test.csv

2023년 03월 04일부터
2023년 03월 31일까지의 데이터

- 예측 대상: price(원/kg)값이 결측치(NaN)로
되어 있으며, 학습된 모델로 예측값을 채울 것

제주 특산물 가격 예측 AI 경진대회

1. 라이브러리 임포트 및 버전 확인

```
import pandas as pd
import numpy as np
import datetime
import random
import os
import sys
import holidays

import sklearn
from sklearn.ensemble import VotingRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_squared_error

import xgboost
from xgboost import XGBRegressor
import catboost
from catboost import CatBoostRegressor
```

```
print(f"파이썬 버전 : {sys.version}")
print(f"pandas 버전 : {pd.__version__}")
print(f"numpy 버전 : {np.__version__}")
print(f"sklearn 버전 : {sklearn.__version__}")
print(f"xgboost 버전 확인 : {xgboost.__version__}")
print(f"catboost 버전 : {catboost.__version__}")
```

제주 특산물 가격 예측 AI 경진대회

2. 시드 고정 (재현성 확보)

: 분석 과정의 무작위성(랜덤성)을 통제하여, 코드를 언제, 어디서 실행하더라도 동일한 결과가 나오도록 재현성을 확보한다.

```
def seed_everything(seed: int = 2024):  
    random.seed(seed)  
    np.random.seed(seed)  
    os.environ["PYTHONHASHSEED"] = str(seed)  
    seed_everything(2024)
```

3. 데이터 로드 및 구조 확인

: 분석에 필요한 훈련 및 테스트 데이터를 불러오고, 훈련 데이터의 초기 구조와 내용을 확인한다.

```
train = pd.read_csv("./train.csv")  
test = pd.read_csv("./test.csv")  
train
```

	ID	timestamp	item	corporation	location	supply(kg)	price(원/kg)
0	TG_A_J_20190101	2019-01-01	TG	A	J	0.0	0.0
1	TG_A_J_20190102	2019-01-02	TG	A	J	0.0	0.0
2	TG_A_J_20190103	2019-01-03	TG	A	J	60601.0	1728.0
3	TG_A_J_20190104	2019-01-04	TG	A	J	25000.0	1408.0
4	TG_A_J_20190105	2019-01-05	TG	A	J	32352.0	1250.0
...	...	...	...	...	...	...	...
59392	RD_F_J_20230227	2023-02-27	RD	F	J	452440.0	468.0
59393	RD_F_J_20230228	2023-02-28	RD	F	J	421980.0	531.0
59394	RD_F_J_20230301	2023-03-01	RD	F	J	382980.0	574.0
59395	RD_F_J_20230302	2023-03-02	RD	F	J	477220.0	523.0
59396	RD_F_J_20230303	2023-03-03	RD	F	J	427520.0	529.0

59397 rows × 7 columns

제주 특산물 가격 예측 AI 경진대회

4. 데이터 전처리 및 특성 추출

1) 전처리 시작 및 데이터 합치기

```
def pre_all(train, test):  
    print(f"전처리 전 train 크기 : {train.shape}")  
    print(f"전처리 전 test 크기 : {test.shape}")  
    print("=====전처리 중=====")  
  
    # 합쳐서 전처리하기  
    train["timestamp"] = pd.to_datetime(train["timestamp"])  
    test["timestamp"] = pd.to_datetime(test["timestamp"])  
    df = pd.concat([train, test]).reset_index(drop = True)  
  
    df.rename(columns={'supply(kg)': 'supply', 'price(원/kg)': 'price'}, inplace=True)
```

1. timestamp 열을 datetime 객체로 변환
(문자열 -> 날짜/시간 형식)

2. 훈련 데이터와 테스트 데이터를
수직으로 합쳐서(concat)
df 데이터프레임 생성

2) 기본적인 시간 관련 특성 추출

```
#년/월/일 추가  
df['year']=df['timestamp'].dt.year  
df['month']=df['timestamp'].dt.month  
df['day']=df['timestamp'].dt.day  
  
#요일 추가  
df['week_day']=df['timestamp'].dt.weekday
```

제주 특산물 가격 예측 AI 경진대회

3) 복합 시계열 특성 추출 및 인코딩

```
# 년-월 변수 추가 : year-month의 형태, 개월단위 누적값
le = LabelEncoder()
df["year_month"] = df["timestamp"].map(lambda x :str(x.year) + "-" +str(x.month))

# 라벨 인코딩
df["year_month"] = le.fit_transform(df["year_month"])

# 주차 변수 추가
df["week"] = df["timestamp"].map(lambda x: datetime.datetime(x.year, x.month, x.day).isocalendar()[1])
```

'연도-월' 형태의
새로운 문자열 특성
year_month 생성

생성된 year_month을
LabelEncoder를 사용하여
0부터 시작하는
순서형 숫자로 변환

timestamp를 이용해
해당 날짜의 주차를 추출
(isocalendar() [1] 이
주차를 나타냄)

제주 특산물 가격 예측 AI 경진대회

4) 주차(Week) 데이터 보정 및 공휴일 변수 추가

```
# 주차 누적값
week_list=[]
for i in range(len(df['year'])):
    if df['year'][i] == 2019:
        week_list.append(int(df['week'][i]))
    elif df['year'][i] == 2020:
        week_list.append(int(df['week'][i])+52)
    elif df['year'][i] == 2021:
        week_list.append(int(df['week'][i])+52+53)
    elif df['year'][i] == 2022:
        week_list.append(int(df['week'][i])+52+53+53)
    elif df['year'][i] == 2023:
        week_list.append(int(df['week'][i])+52+53+53+52)
df['week_num']= week_list
```

- isocalendar() 함수는 연도 시작 주차 계산 시 오류를 발생시키므로, 각 연도에 따라 추가적인 주차 수 (52주 혹은 53주)를 더해 연속적인 주차 번호를 생성

```
# datetime 패키지에서 19년 12월 마지막주가 첫째주로 들어가는거 발견하여 수정
df.loc[df['timestamp']=='2019-12-30','week_num']=52
df.loc[df['timestamp']=='2019-12-31','week_num']=52

# 공휴일 변수 추가
def make_holi(x):
    kr_holi = holidays.KR()

    if x in kr_holi:
        return 1
    else:
        return 0

df["holiday"] = df["timestamp"].map(lambda x : make_holi(x))
```

- datetime 패키지에서 2019년 12월 마지막 주가 다음 해 첫 주로 들어가는 오류가 발생하므로 특정 날짜의 week_num값을 52로 직접 수정
- holidays.KR() 라이브러리를 사용해 공휴일이면 1, 아니면 0을 반환

제주 특산물 가격 예측 AI 경진대회

5) 데이터 재분리 및 최종 확인

```
# train, test 분리하기
train = df[~df["price"].isnull()].sort_values("timestamp").reset_index(drop = True)
test = df[df["price"].isnull()].sort_values("timestamp").reset_index(drop=True)

print(f"전처리 후 train 크기 : {train.shape}")
print(f"전처리 후 test 크기 : {test.shape}")

return train, test
train_pre, test_pre = pre_all(train, test)
```

train_pre

	ID	timestamp	item	corporation	location	supply	price	year	month	day	week_day	year_month	week	week_num	holiday
0	TG_A_J_20190101	2019-01-01	TG	A	J	0.0	0.0	2019	1	1	1	0	1	1	1
1	CB_A_S_20190101	2019-01-01	CB	A	S	0.0	0.0	2019	1	1	1	0	1	1	1
2	RD_D_J_20190101	2019-01-01	RD	D	J	0.0	0.0	2019	1	1	1	0	1	1	1
3	BC_D_J_20190101	2019-01-01	BC	D	J	0.0	0.0	2019	1	1	1	0	1	1	1
4	CB_F_J_20190101	2019-01-01	CB	F	J	0.0	0.0	2019	1	1	1	0	1	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
59392	CR_E_S_20230303	2023-03-03	CR	E	S	0.0	0.0	2023	3	3	4	50	9	219	0
59393	BC_A_S_20230303	2023-03-03	BC	A	S	3776.0	2875.0	2023	3	3	4	50	9	219	0
59394	CB_E_J_20230303	2023-03-03	CB	E	J	0.0	0.0	2023	3	3	4	50	9	219	0
59395	BC_D_J_20230303	2023-03-03	BC	D	J	1776.0	3059.0	2023	3	3	4	50	9	219	0
59396	RD_F_J_20230303	2023-03-03	RD	F	J	427520.0	529.0	2023	3	3	4	50	9	219	0

59397 rows × 15 columns

test_pre

	ID	timestamp	item	corporation	location	supply	price	year	month	day	week_day	year_month	week	week_num	holiday
0	TG_A_J_20230304	2023-03-04	TG	A	J	NaN	NaN	2023	3	4	5	50	9	219	0
1	TG_E_S_20230304	2023-03-04	TG	E	S	NaN	NaN	2023	3	4	5	50	9	219	0
2	BC_B_J_20230304	2023-03-04	BC	B	J	NaN	NaN	2023	3	4	5	50	9	219	0
3	TG_E_J_20230304	2023-03-04	TG	E	J	NaN	NaN	2023	3	4	5	50	9	219	0
4	BC_B_S_20230304	2023-03-04	BC	B	S	NaN	NaN	2023	3	4	5	50	9	219	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1087	TG_A_J_20230331	2023-03-31	TG	A	J	NaN	NaN	2023	3	31	4	50	13	223	0
1088	RD_D_J_20230331	2023-03-31	RD	D	J	NaN	NaN	2023	3	31	4	50	13	223	0
1089	CR_D_J_20230331	2023-03-31	CR	D	J	NaN	NaN	2023	3	31	4	50	13	223	0
1090	TG_E_J_20230331	2023-03-31	TG	E	J	NaN	NaN	2023	3	31	4	50	13	223	0
1091	RD_F_J_20230331	2023-03-31	RD	F	J	NaN	NaN	2023	3	31	4	50	13	223	0

1092 rows × 15 columns

제주 특산물 가격 예측 AI 경진대회

5. TG 제외 나머지 농산물

1) 전처리

(1) 이상치(극단값) 식별 및 제거

```
# 극 이상치 제거
tg_idx = train_pre[(train_pre["item"]=="TG") & (train_pre["price"]>20000)].index
rd_idx = train_pre[(train_pre["item"]=="RD") & (train_pre["price"]>5000)].index
bc_idx = train_pre[(train_pre["item"]=="BC") & (train_pre["price"]>8000)].index
cb_idx = train_pre[(train_pre["item"]=="CB") & (train_pre["price"]>2300)].index

train_pre.loc[tg_idx,"price"] = train_pre[(train_pre["item"]=="TG") & (train_pre["price"]!=0)]["price"].mean()
train_pre.loc[rd_idx,"price"] = train_pre[(train_pre["item"]=="RD") & (train_pre["price"]!=0)]["price"].mean()
train_pre.loc[bc_idx,"price"] = train_pre[(train_pre["item"]=="BC") & (train_pre["price"]!=0)]["price"].mean()
train_pre.loc[cb_idx,"price"] = train_pre[(train_pre["item"]=="CB") & (train_pre["price"]!=0)]["price"].mean()
```

: 품목별로 극단값을 식별하고, 해당 품목의 평균값으로 대체

(2) TG(감귤) 제외

```
train_notg = train_pre[train_pre["item"]!="TG"]
test_notg = test_pre[test_pre["item"]!="TG"]
```

제주 특산물 가격 예측 AI 경진대회

(3) 범주형 변수 인코딩

- 문자열로 된 범주형 특성을 숫자 형태로 변환(인코딩)
- get_dummies를 사용하여 원-핫 인코딩 수행

#인코딩

```
Xy = pd.get_dummies(train_notg.sort_values(by = ["timestamp"]).reset_index(drop=True).drop(columns = ["supply"]), columns = ["item", "corporation"])
answer_notg = pd.get_dummies(test_notg.drop(columns = ["timestamp", "supply", "price"]), columns = ["item", "corporation", "location"])
print(Xy.columns)
```

```
Index(['ID', 'timestamp', 'price', 'year', 'month', 'day', 'week_day',
      'year_month', 'week', 'week_num', 'holiday', 'item_BC', 'item_CB',
      'item_CR', 'item_RD', 'corporation_A', 'corporation_B', 'corporation_C',
      'corporation_D', 'corporation_E', 'corporation_F', 'location_J',
      'location_S'],
      dtype='object')
```

제주 특산물 가격 예측 AI 경진대회

2) 모델링 & 훈련예측

(1) 개별 기본 모델(Base Model) 정의

양상블에 사용할 두 가지 강력한 부스팅 모델인 CatBoost와 XGBoost의 하이퍼파라미터를 설정하고 정의

```
cat = CatBoostRegressor(random_state = 2024,  
                        n_estimators = 1000,  
                        learning_rate = 0.01,  
                        depth = 10,  
                        l2_leaf_reg = 3,  
                        metric_period = 1000)  
  
xgb = XGBRegressor(n_estimators = 1000, random_state = 2024, learning_rate = 0.01, max_depth = 10)
```

(2) 보팅 회귀 모델 정의 및 학습

```
# voting  
vote_model = VotingRegressor(  
    estimators=[("cat", cat), ("xgb", xgb)]  
)  
  
vote_model.fit(Xy.drop(columns = ["timestamp", "ID", "price"]), Xy["price"])
```

제주 특산물 가격 예측 AI 경진대회

(3) 최종 예측 및 결과 처리

학습된 앙상블 모델을 사용하여 테스트 데이터의 가격을 예측하고, 예측 결과에 대한 후처리 수행

```
pred = vote_model.predict(answer_notg.drop(columns = ["ID"]))
for idx in range(len(pred)):
    if pred[idx]<0:
        pred[idx]= 0
answer_notg["answer"] = pred

answer_notg[["ID","answer"]]
```

- 가격이 음수일 수는 없으므로 예측값이 0보다 작다면, 해당 예측값을 0으로 강제 보정함

	ID	answer
2	BC_B_J_20230304	2376.672901
4	BC_B_S_20230304	97.678967
6	BC_C_J_20230304	2385.800075
7	BC_A_S_20230304	2763.041513
10	BC_D_J_20230304	2771.454688
...	...	...
1085	RD_D_S_20230331	476.193654
1086	CR_C_J_20230331	1696.957199
1088	RD_D_J_20230331	387.570045
1089	CR_D_J_20230331	1818.944640
1091	RD_F_J_20230331	492.058492

제주 특산물 가격 예측 AI 경진대회

6. TG(1)

1) 전처리

(1) 초기 함수 호출 및 공휴일 특성 수정

```
train_pre, test_pre = pre_all(train, test)

# 공휴일이지만 안쉬는 날 제외하기
no_holi = list(train_pre[(train_pre["item"]=="TG") & (train_pre["holiday"]==1) & (train_pre["price"]!=0)].groupby("timestamp").count().reset_index()[
noholi_idx = train_pre[train_pre["timestamp"].isin(no_holi)][["holiday"]].index
for idx in noholi_idx:
    train_pre.loc[idx, "holiday"] = 0
```

품목이 'TG'이고,
holiday가 1(공휴일)이며,
가격이 0이 아닌
데이터 선택

Timestamp별로
그룹화하여
날짜 목록을 추출하고
no_holi에 저장

가격 거래가 있었으므로,
공휴일 특성을 0으로 처리해
모델이 해당 날짜를
공휴일로 인식하지 않게 함

제주 특산물 가격 예측 AI 경진대회

(2) TG 데이터 인코딩 및 최종 준비

```
# train 및 test 시간 순서로 정렬하기
train_tg = train_pre[train_pre["item"] == "TG"].sort_values(by = ["timestamp"]).reset_index(drop= True)
test_tg = test_pre[test_pre["item"] == "TG"].sort_values(by = ["timestamp"]).reset_index(drop= True)

Xy = pd.get_dummies(train_tg, columns = [ "item", "corporation", "location"]).drop(columns = ["supply"])
answer_tg1 = pd.get_dummies(test_tg, columns = [ "item", "corporation", "location"]).drop(columns = ["timestamp", "supply", "price"])
print(f"train의 컬럼 : {Xy.columns}")
print(f"test의 컬럼 : {answer_tg1.columns}")
Xy["price"] = np.sqrt(Xy["price"])
```

전처리 전 train 크기 : (59397, 7)

전처리 전 test 크기 : (1092, 5)

=====전처리 중=====

전처리 후 train 크기 : (59397, 15)

전처리 후 test 크기 : (1092, 15)

train의 컬럼 : Index(['ID', 'timestamp', 'price', 'year', 'month', 'day', 'week_day',
 'year_month', 'week', 'week_num', 'holiday', 'item_TG', 'corporation_A',
 'corporation_B', 'corporation_C', 'corporation_D', 'corporation_E',
 'location_J', 'location_S'],
 dtype='object')

test의 컬럼 : Index(['ID', 'year', 'month', 'day', 'week_day', 'year_month', 'week',
 'week_num', 'holiday', 'item_TG', 'corporation_A', 'corporation_B',
 'corporation_C', 'corporation_D', 'corporation_E', 'location_J',
 'location_S'],
 dtype='object')

예측 대상의 분포를 정규화하여
모델의 성능을 향상시키기 위해
price에 제곱근 변환 적용

제주 특산물 가격 예측 AI 경진대회

2) 모델링& 훈련예측

```
## 모델 정의
cat = CatBoostRegressor(random_state = 2024,
                        n_estimators = 1000,
                        learning_rate = 0.01,
                        depth = 10,
                        l2_leaf_reg = 3,
                        metric_period = 1000)

xgb = XGBRegressor(n_estimators = 1000, random_state = 2024, learning_rate = 0.01, max_depth = 10)

# voting
vote_model = VotingRegressor(
    estimators=[("cat", cat), ("xgb", xgb)]
)

vote_model.fit(Xy.drop(columns = ["timestamp", "ID", "price"]), Xy["price"])

pred = vote_model.predict(answer_tg1.drop(columns = ["ID"]))
for idx in range(len(pred)):
    if pred[idx]<0:
        pred[idx]= 0
    answer_tg1["answer"] = np.power(pred,2)

answer_tg1[["ID", "answer"]]
```

	ID	answer
0	TG_A_J_20230304	2751.208449
1	TG_E_S_20230304	3381.528438
2	TG_E_J_20230304	529.638514
3	TG_D_S_20230304	3764.708357
4	TG_D_J_20230304	407.141038
...	...	...
275	TG_D_J_20230331	1870.228988
276	TG_D_S_20230331	5046.532954
277	TG_A_S_20230331	5182.915243
278	TG_E_S_20230331	4690.096292
279	TG_E_J_20230331	1526.626858

제주 특산물 가격 예측 AI 경진대회

7. TG (2)

: 단일 CatBoost + 새로운 튜닝 파라미터를 활용하여 일반화를 위한 추가 모델링

1) 전처리

```
train_tg2 = train_pre[train_pre["item"] == "TG"]
test_tg2 = test_pre[test_pre["item"] == "TG"]

Xy2 = pd.get_dummies(train_tg2.sort_values(by = ["timestamp", "corporation", "location"]).reset_index(drop=True).drop(columns = ["item", "supply"]
answer_tg2 = pd.get_dummies(test_tg2.drop(columns = ["timestamp", "supply", "price", "item"]), columns = [ "corporation", "location"])
print(Xy2.columns)

# 종속변수 루트값
Xy2["price"] = np.sqrt(Xy2["price"])

Index(['ID', 'timestamp', 'price', 'year', 'month', 'day', 'week_day',
       'year_month', 'week', 'week_num', 'holiday', 'corporation_A',
       'corporation_B', 'corporation_C', 'corporation_D', 'corporation_E',
       'location_J', 'location_S'],
      dtype='object')
```

제주 특산물 가격 예측 AI 경진대회

2) 모델링& 훈련예측

```
n_estimators=1000
lrs = 0.05
max_depths = 10
l2_leaf_reg = 3

cat = CatBoostRegressor(random_state = 2024,
                        n_estimators = n_estimators,
                        learning_rate = lrs,
                        depth = max_depths,
                        l2_leaf_reg = l2_leaf_reg,
                        metric_period = 1000)

cat.fit(Xy2.drop(columns = ["timestamp", "ID", "price"]), Xy2["price"])

pred2 = cat.predict(answer_tg2.drop(columns = ["ID"]))
for idx in range(len(pred2)):
    if pred2[idx]<0:
        pred2[idx]= 0
answer_tg2["answer"] = np.power(pred2,2)

answer_tg2[["ID", "answer"]]
```

	ID	answer
0	TG_A_J_20230304	2742.172780
1	TG_E_S_20230304	3354.003496
3	TG_E_J_20230304	760.581106
5	TG_D_S_20230304	3769.615969
8	TG_D_J_20230304	53.451592
...	...	...
1074	TG_D_S_20230331	4727.895634
1077	TG_A_S_20230331	5403.967583
1079	TG_E_S_20230331	4715.201735
1087	TG_A_J_20230331	6813.935154
1090	TG_E_J_20230331	2810.642810

제주 특산물 가격 예측 AI 경진대회

8. TG 앙상블

```
total1 = pd.concat([answer_tg1[["ID", "answer"]], answer_notg[["ID", "answer"]]])  
total2 = pd.concat([answer_tg2[["ID", "answer"]], answer_notg[["ID", "answer"]]])
```

```
# TG 앙상블 (평균)
```

```
df = pd.merge(total1, total2, how = "inner", on="ID")  
df["answer"] = (df["answer_x"] + df["answer_y"]) / 2  
df["item"] = df["ID"].map(lambda x : x.split("_")[0])
```

- 두 개의 독립적인 최종 예측 세트(total1, total2)를 ID 기준으로 합함
- 예측값들을 단순 평균하여 최종적으로 가장 안정적이고 일반화된 모델의 예측 결과를 도출함

제주 특산물 가격 예측 AI 경진대회

8. 후처리

```
# 후처리
## 전체 min값 | 3월의 min값 확인
df.loc[(df['item']=='TG')&(df['answer']<400), 'answer'] = 0 # 551 #3월 675
df.loc[(df['item']=='CB')&(df['answer']<50), 'answer'] = 0 # 162 # 3월 200
df.loc[(df['item']=='RD')&(df['answer']<10), 'answer'] = 0 # 50 # 3월 124
df.loc[(df['item']=='CR')&(df['answer']<150), 'answer'] = 0 # 250 # 3월 450
df.loc[(df['item']=='BC')&(df['answer']<100), 'answer'] = 0 # 205 3월 205.0
```

```
df = df.drop(columns = ["answer_x", "answer_y", "item"])
df
```

	ID	answer
0	TG_A_J_20230304	2746.690615
1	TG_E_S_20230304	3367.765967
2	TG_E_J_20230304	645.109810
3	TG_D_S_20230304	3767.162163
4	TG_D_J_20230304	0.000000
...	...	...
1087	RD_D_S_20230331	476.193654
1088	CR_C_J_20230331	1696.957199
1089	RD_D_J_20230331	387.570045
1090	CR_D_J_20230331	1818.944640
1091	RD_F_J_20230331	492.058492

- 품목별로 예측값(answer)이 특정 최소 기준보다 낮을 경우, 해당 값을 0으로 강제 보정함
- answer_x, answer_y, item 열을 최종 데이터프레임 df에서 제거함

제주 특산물 가격 예측 AI 경진대회

9. 차별점, 배울점

1. 데이터 이질성에 대한 전략적 대응

감귤의 가격 변동이 다른 채소들과 근본적으로 다르다는 점을 인식하고, 감귤 데이터만 따로 분리하여 해당 품목의 고유 패턴에 최적화된 모델링을 수행함

⇒ 모든 품목에 동일한 기준을 적용하지 않고, 품목별 가격 범위를 고려한 맞춤형 이상치 처리 기준을 설정하여 데이터의 품질을 극대화함

2. 데이터 배경 이해의 중요성

실제 유통 데이터를 통해 공휴일 데이터를 수정함

⇒ 분석의 오류를 줄이고
최종 예측 성능을 안정적으로 높임

기업 성공 확률 해커톤

0. 라이브러리

```
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd

from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestRegressor

import plotly.express as px
import plotly.graph_objects as go
import seaborn as sns
import matplotlib.pyplot as plt
```

기업 성공 확률 해커톤

1. 데이터 로드

```
target = '성공확률'
```

```
df = pd.read_csv('./train.csv')  
df = df.drop(columns=['ID'])  
df_test = pd.read_csv('./test.csv')  
df_test = df_test.set_index('ID')
```

	설립 연도	국가	분야	투자 단계	직원 수	인 수 여 부	상 장 여 부	고객 수(백 만명)	총 투자 금(억 원)	연매출 (억원)	SNS 팔 로워 수 (백만 명)	기업가 치(백 억원)	성 공 확 률
0	2009	CT005	이커 머스	Series A	4126.0	No	No	56.0	3365.0	4764.0	4.71	NaN	0.3
1	2023	CT006	핀테 크	Seed	4167.0	Yes	No	80.0	4069.0	279.0	1.00	2500- 3500	0.8
2	2018	CT007	기술	Series A	3132.0	Yes	Yes	54.0	6453.0	12141.0	4.00	3500- 4500	0.5
3	2016	CT006	NaN	Seed	3245.0	Yes	Yes	NaN	665.0	10547.0	2.97	NaN	0.7
4	2020	CT002	에듀 테크	Seed	1969.0	No	Yes	94.0	829.0	9810.0	1.00	1500- 2500	0.1

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4376 entries, 0 to 4375
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	설립연도	4376 non-null	int64
1	국가	4376 non-null	object
2	분야	3519 non-null	object
3	투자단계	4376 non-null	object
4	직원 수	4202 non-null	float64
5	인수여부	4376 non-null	object
6	상장여부	4376 non-null	object
7	고객수(백만명)	3056 non-null	float64
8	총 투자금(억원)	4376 non-null	float64
9	연매출(억원)	4376 non-null	float64
10	SNS 팔로워 수(백만명)	4376 non-null	float64
11	기업가치(백억원)	3156 non-null	object
12	성공확률	4376 non-null	float64

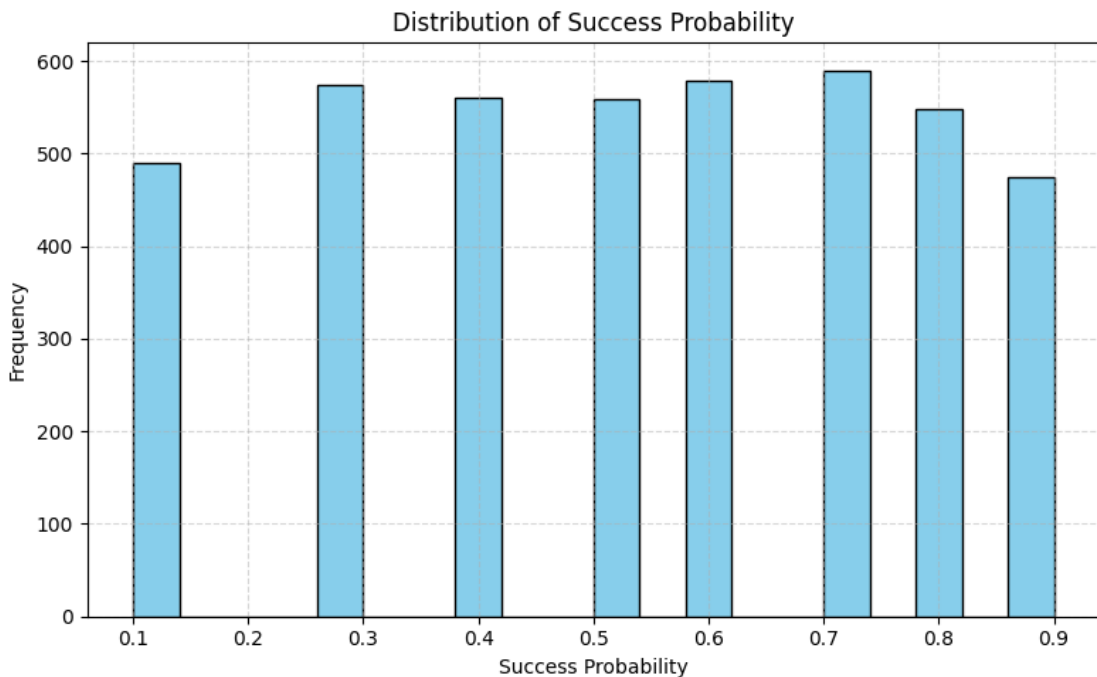
```
dtypes: float64(6), int64(1), object(6)
```

```
memory usage: 444.6+ KB
```

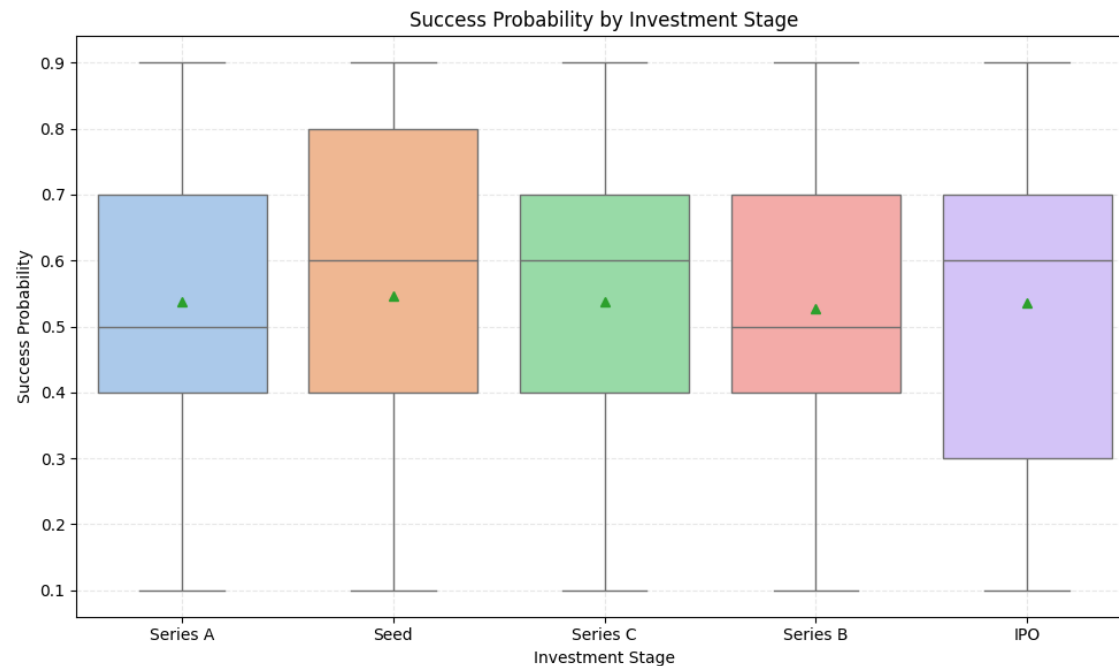
기업 성공 확률 해커톤

2. EDA

```
plt.figure(figsize=(8, 5))
plt.hist(df['성공확률'], bins=20, color='skyblue', edgecolor='black')
plt.title('Distribution of Success Probability')
plt.xlabel('Success Probability')
plt.ylabel('Frequency')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.boxplot(x='투자단계', y='성공확률', data=df, showmeans=True, palette='pastel')
plt.title('Success Probability by Investment Stage')
plt.xlabel('Investment Stage')
plt.ylabel('Success Probability')
plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```

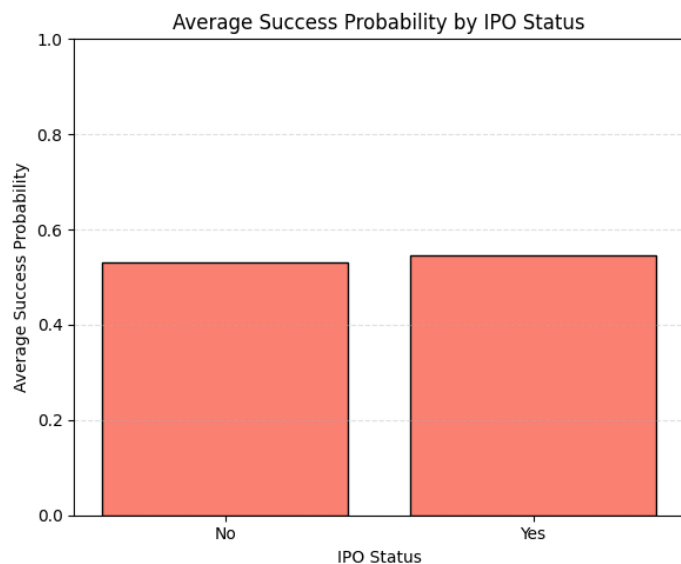


기업 성공 확률 해커톤

2. EDA

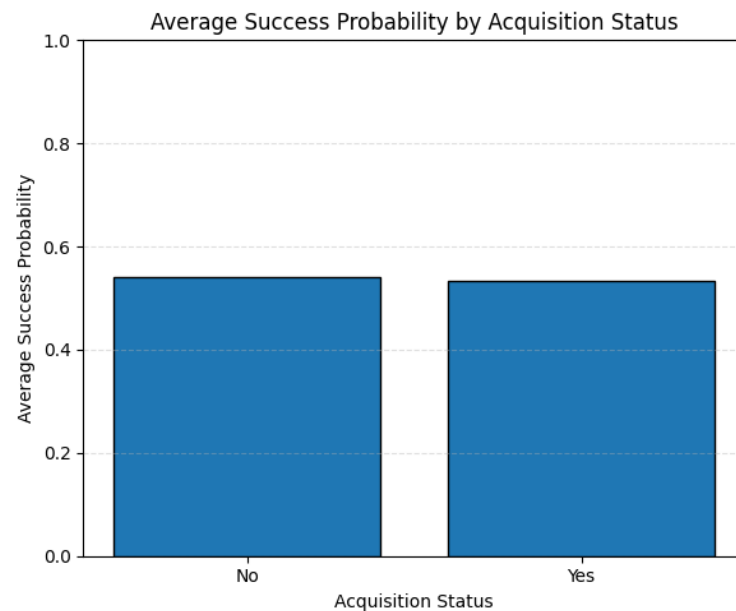
```
means = df.groupby('상장여부')['성공확률'].mean().reset_index()

plt.figure(figsize=(6, 5))
plt.bar(means['상장여부'], means['성공확률'], color='salmon', edgecolor='black')
plt.title('Average Success Probability by IPO Status')
plt.xlabel('IPO Status')
plt.ylabel('Average Success Probability')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



```
means = df.groupby('인수여부')['성공확률'].mean().reset_index()

plt.figure(figsize=(6, 5))
plt.bar(means['인수여부'], means['성공확률'], edgecolor='black')
plt.title('Average Success Probability by Acquisition Status')
plt.xlabel('Acquisition Status')
plt.ylabel('Average Success Probability')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



기업 성공 확률 해커톤

3. 전처리: 오브젝트 타입 피처(기업가치)를 실수형태로 변환

기업가
치(백
억원)

NaN

2500-
3500

3500-
4500

NaN

1500-
2500

```
def convert_기업가치(value):  
    if pd.isna(value):  
        return np.nan  
    if value == '6000이상':  
        return 6500  
    if '-' in value:  
        start, end = value.split('-')  
        return (int(start) + int(end)) / 2  
    return np.nan  
df['기업가치(백억원)'] = df['기업가치(백억원)'].apply(convert_기업가치)  
df_test['기업가치(백억원)'] = df_test['기업가치(백억원)'].apply(convert_기업가치)
```

기업 성공 확률 해커톤

3. 전처리: 결측치 처리

```
df['분야'] = df['분야'].fillna('Null')
df_test['분야'] = df_test['분야'].fillna('Null')

# float 형식의 경우 그룹의 평균 값으로 대체
group_cols = ['설립연도', '인수여부', '상장여부']
for col in ['직원 수', '고객수(백만명)', '기업가치(백억원)']:
    df[col] = df.groupby(group_cols)[col].transform(
        lambda x: x.fillna(x.mean())
    )
    df_test[col] = df_test.groupby(group_cols)[col].transform(
        lambda x: x.fillna(x.mean())
    )
for col in ['직원 수', '고객수(백만명)', '기업가치(백억원)']:
    df[col] = df[col].fillna(df[col].mean())
    df_test[col] = df_test[col].fillna(df_test[col].mean())
```



범주형 변수로 그룹화하여 각 그룹별
평균값을 채움



그룹별 평균으로 채워지지 않은 경우
전체 평균으로 채움

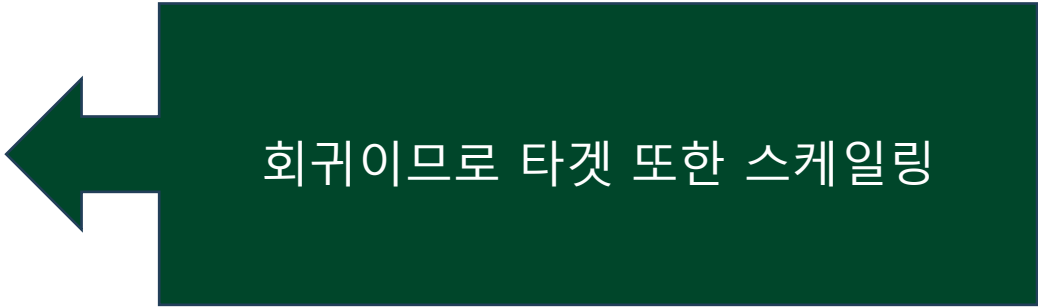
기업 성공 확률 해커톤

3. 전처리: 스케일링 및 라벨 인코딩

```
cols = df.drop(columns=target).select_dtypes('float')
ss = StandardScaler()
for col in cols:
    df[col] = ss.fit_transform(df[[col]])
    df_test[col] = ss.transform(df_test[[col]])

ss = StandardScaler()
df[target] = ss.fit_transform(df[[target]])

cols = df.select_dtypes(exclude=['float'])
le = LabelEncoder()
for col in cols:
    df[col] = le.fit_transform(df[[col]])
    df_test[col] = le.transform(df_test[[col]])
```



회귀이므로 타겟 또한 스케일링

기업 성공 확률 해커톤

4. 피쳐 중요도 확인: 추후 낮은 중요도 피쳐들을 제거해보며 성능 확인하기

```
train_X = df.drop(columns=[target])
train_y = df[target]
train_X = train_X

model = RandomForestRegressor(random_state=10)
model.fit(train_X, train_y)
```

```
pd.DataFrame(model.feature_importances_,
              model.feature_names_in_,
              columns=['importance']).sort_values(by='importance',
                                                  ascending=False)
```

	importance
고객수(백만명)	0.079404
직원 수	0.078192
직원당_투자금	0.074863
투자금대비_기업가치	0.074203
고객당_연매출	0.073944
직원당_연매출	0.073936
SNS 팔로워 수(백만명)	0.073913
연매출대비_기업가치	0.073496
총 투자금(억원)	0.072051
연매출(억원)	0.068138
설립연도	0.062168
국가	0.045559
분야	0.043569

기업가치(백억원)	0.039044
투자단계_인코딩	0.023632
투자단계	0.023020
상장여부	0.010489
인수여부	0.010378

기업 성공 확률 해커톤

5. Kfold(fold=5)교차 검증: 칼럼 드랍 및 하이퍼 파라미터 튜닝

```
# 데이터 준비
train_X = df.drop(columns=[target])
train_y = df[target]
train_X = train_X.drop(columns=['국가', '분야', '설립연도', '투자단계'])

# KFold 설정
kf = KFold(n_splits=5, shuffle=True, random_state=42)
fold = 1
mae_list = []

# 모델 파라미터
params = {
    'n_estimators': 950,
    'max_depth': 35,
    'max_features': 'sqrt',
    'criterion': 'friedman_mse',
    'bootstrap': False
}
```

```
# KFold 루프
for train_index, val_index in kf.split(train_X):
    X_train, X_val = train_X.iloc[train_index], train_X.iloc[val_index]
    y_train, y_val = train_y[train_index], train_y[val_index]

    model = RandomForestRegressor(**params, random_state=10)
    model.fit(X_train, y_train)

    preds = model.predict(X_val)
    preds = ss.inverse_transform(preds.reshape(-1, 1)).flatten()
    y_val_true = ss.inverse_transform(y_val.values.reshape(-1, 1)).flatten()

    mae = mean_absolute_error(y_val_true, preds)
    mae_list.append(mae)

print(f"[Fold {fold}] MAE: {mae:.5f}")
```

Fold1 MAE: 0.19284

Fold2 MAE: 0.19495

Fold3 MAE: 0.19202

Fold4 MAE: 0.19248

Fold5 MAE: 0.19136

평균 MAE: 0.19273

기업 성공 확률 해커톤

6. 모델링: 최종 예측값 추출

```
train_X = df.drop(columns=[target])
train_y = df[target]
train_X = train_X.drop(columns=['국가', '분야', '설립연도', '투자단계'])
df_test = df_test.drop(columns=['국가', '분야', '설립연도', '투자단계'])

params = {
    'n_estimators': 950
    , 'max_depth': 35
    , 'max_features': 'sqrt'
    , 'criterion': 'squared_error'
    , 'bootstrap': False
}

model = RandomForestRegressor(**params, random_state=10)
model.fit(train_X, train_y)
pred = model.predict(df_test)
df_test[target] = pred

df_test[target] = ss.inverse_transform(df_test[[target]])
```

7. 후처리

```
df_test[target] = df_test[target] - 0.0375
df_test[target] = round(df_test[target], 1)
```

8. 결과 저장

```
df_test.reset_index()[['ID', target]].to_csv(f"./result.csv", index=False)
```

기업 성공 확률 해커톤

❖ 차별점 및 배울점

- 각 피처들을 이용해 다양한 파생변수를 생성
- 범주형 변수로 그룹화한 평균값으로 결측치를 채움
- 피처 중요도를 파악해 상대적으로 불필요한 피처들을 제거
- 예측값의 과대추정 보정

주식 종료 가격 예측

데이터 설명

- Stock_List.csv: 종목 번호 데이터
 - 종목명: 주식 종목 명
 - 종목코드: 주식 종목 코드번호
 - 상장시장: 주식 종목 상장 시장 (KOSPI or KOSDAQ)

☐	종목명	종목코드	상장시장
1	삼성전자	5930	KOSPI
2	SK하이닉스	660	KOSPI
3	NAVER	35420	KOSPI
4	카카오	35720	KOSPI
5	삼성바이오로직스	207940	KOSPI

전처리 방법

- 시계열 분해를 통해 종가 추세와 계절성 데이터를 추가
- 종가에 대하여 5일, 20일, 60일, 120일, 240일 이동평균을 추가
- 최종적으로 일주일씩 (business day 기준으로 5일씩) lag features를 생성

주식 종료 가격 예측

전처리

기본 날짜 프레임 생성

- 2011년 1월 1일부터 2021년 9월 17일까지의 영업일(business day) 기준 날짜 생성
- 연도, 월, 주, 일, 요일, 월별 주차 등의 시간 관련 파생 변수 추가

주가 데이터 수집 및 병합

- 각 종목 코드에 대해 주가 데이터 불러오기
- 날짜 기준으로 병합하고, 공휴일 등 결측치는 전날 값으로 보간
- 상장 전 기간의 결측치는 제거

시계열 분해 (trend / seasonal 분리)

- 종가 데이터를 시계열 분해
- 추세와 계절성 값을 새로운 칼럼으로 추가

주식 종료 가격 예측

이동평균(rolling mean) 추가

- 종가에 대해 5일, 20일, 60일, 120일, 240일 이동평균선 추가
- 단기 / 중기 / 장기 추세를 반영한 특징 생성

```
# 이동평균 추가하는 함수 정의
def add_rolling_mean(df:pd.DataFrame, col_name:str, window_list:list)->pd.DataFrame:
    for window in window_list:
        df[col_name + f'_rolling{window}'] = df[col_name].rolling(window).mean().fillna(method = 'bfill')
    return df
```

Lag(시차) 피쳐 생성

- 주요 수치형 칼럼들을 5일 전 값으로 이동
- 과거 정보(5영업일 전)를 현재 예측에 활용할 수 있도록 변환

```
# 컬럼값을 lag하는 함수 정의
def lag_features(df:pd.DataFrame, col_list:list, lag_num:int)->pd.DataFrame:
    for col in col_list:
        df[col] = df[col].shift(lag_num).fillna(method = 'ffill').fillna(method = 'bfill')
    return df
```

정리 및 저장

- 인덱스 초기화 후, 최종 전처리된 종목별 DataFrame을 stock_df_dict[stock_code]에 저장

주식 종료 가격 예측

모델링 (XGBoost)

SEED 고정

- 모델의 재현성을 위해 랜덤 시드를 고정

NMAE 함수 정의

- Normalized Mean Absolute Error(정규화된 평균 절대 오차) 계산 함수
- 예측 오차를 실제값으로 나눠서 상대적인 오차 비율(%)로 평가

```
# NMAE 함수 정의
def NMAE(true, pred):
    return np.mean(abs(true - pred) / true) * 100
```

반복 학습 구조

- stock_df_dict에 있는 모든 종목에 대해 반복 학습 수행
- 각 종목별 모델을 따로 학습하고, 예측 결과를 저장

주식 종료 가격 예측

데이터 분할 (Public / Private 구간)

- 각 종목별 데이터를 시계열 순서로 분할
- Public
 - train: 전체 중 마지막 15일 제외
 - test: 마지막 15일 중 최근 10~15일 구간
 - valid: train의 마지막 5일
- Private
 - train: 전체 중 마지막 5일 제외 구간
 - test: 마지막 5일
 - valid: train의 마지막 5일

```
# split train, valid, test (public)
train = stock_df_dict[stock_code][: -15]
test = stock_df_dict[stock_code][ -15: -10]
```

```
X_train = train[: -5].drop(columns = ['Date', 'Close'])
y_train = train[: -5]['Close']
X_valid = train[ -5:].drop(columns = ['Date', 'Close'])
y_valid = train[ -5:] ['Close']
X_test = test.drop(columns = ['Date', 'Close'])
```

주식 종료 가격 예측

모델 학습

- XGBoost 회귀 모델 사용
- 파라미터 설정
 - n_estimators=10000: 최대 트리 개수
 - learning_rate=0.05: 학습률 (낮게→천천히 학습)
 - tree_method='gpu_hist': GPU 가속
 - early_stopping_rounds=100: 검증 손실이 100회 연속 개선되지 않으면 조기 종료
 - eval_metric='mae': 평가 지표로 MAE 사용

```
# xgboost
xgb = XGBRegressor(n_estimators= 10000, learning_rate = 0.05, tree_method = 'gpu_hist', random_state = SEED)
xgb.fit(X_train, y_train, eval_set = [(X_valid, y_valid)], early_stopping_rounds = 100, eval_metric = 'mae', verbose = 100)

xgb_oof = xgb.predict(X_valid)
xgb_pred = xgb.predict(X_test)
xgb_pred_dict[stock_code] = xgb_pred
```

주식 종료 가격 예측

예측 및 성능 측정

- xgb_oof: 검증 구간(최근 5일)의 예측값
- xgb_pred: 테스트 구간의 예측값
- 각 종목별 예측 결과를 xgb_pred_dict에 저장

NMAE 계산 및 출력

- 검증 구간의 NMAE 점수를 계산해 모델 성능을 평가
- 각 종목의 점수를 리스트에 저장

최종 평균 성능 출력

- 모든 종목의 검증 결과를 평균 내어, 모델의 전체적인 성능을 요약

```
0% |          | 0/376 [00:00<?, ?it/s]
[종목코드 000060 종목명 메리츠화재 학습 및 예측]
[07:04:03] WARNING: /workspace/src/objective/regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[0]      validation_0-mae:30377.5
Will train until validation_0-mae hasn't improved in 100 rounds.
[100]    validation_0-mae:4155.55
[200]    validation_0-mae:3347.88
[300]    validation_0-mae:3420.38
Stopping. Best iteration:
[225]    validation_0-mae:3345.55
```

주식 종료 가격 예측

날짜별 각 종목의 예측 종가

	Day	000060	000080	000100	000120	000150	000240	000250	000270	000660	000670	000720	000810
0	2021-08-30	24925.125000	31336.667969	61190.394531	167265.375000	88352.039062	17018.898438	54904.292969	81078.289062	103442.562500	650774.5625	48399.035156	221944.562500
1	2021-08-31	24916.582031	32029.447266	60760.246094	168107.171875	88352.039062	16599.865234	55507.550781	81603.750000	103442.562500	646966.4375	48951.753906	221944.562500
2	2021-09-01	24943.074219	32538.402344	61465.183594	167788.953125	89079.085938	16599.865234	55815.164062	81697.585938	103442.562500	659527.5000	50402.152344	227437.515625
3	2021-09-02	24966.750000	32546.369141	61074.714844	167904.203125	89593.171875	16599.865234	54377.402344	81697.585938	103442.562500	661566.7500	50590.671875	227437.515625
4	2021-09-03	24667.744141	32907.648438	61688.386719	167941.000000	89853.914062	16599.865234	55540.113281	81920.343750	103442.562500	672899.0000	51134.902344	227296.015625
5	2021-09-13	28108.398438	35277.746094	63838.695312	168976.609375	93227.023438	16728.806641	51587.750000	85537.054688	104308.554688	730594.0000	54569.691406	228319.734375
6	2021-09-14	28310.734375	35186.847656	64073.015625	169314.765625	91899.554688	16742.607422	51587.750000	85929.906250	104308.554688	727685.2500	53703.382812	225699.046875
7	2021-09-15	28310.734375	34956.558594	63490.070312	169192.531250	91769.976562	16728.806641	51409.890625	84833.593750	105262.070312	727025.7500	53399.714844	225844.593750
8	2021-09-16	28414.166016	35032.687500	63286.054688	169239.187500	91047.804688	16742.607422	51587.750000	84456.531250	104308.554688	723455.9375	53366.292969	225530.546875
9	2021-09-17	27893.125000	34902.191406	63772.664062	169190.031250	91826.617188	16742.607422	51587.750000	84346.375000	105262.070312	728129.5625	53738.757812	227196.031250

주식 종료 가격 예측

차별점 및 배울점

종목별 독립 모델링

- 모든 종목을 하나로 묶어 모델링하지 않고, 종목별로 개별 데이터프레임+개별 모델을 학습
- 종목별 특성이 반영되어 맞춤형 예측 가능

시계열 기반 전처리

- rolling mean과 lag features를 활용
- 종가 trend와 seasonal 분해 후 피쳐화
- 단순 회귀/트리 모델에 시계열 패턴을 반영

결측치 처리

- 공휴일, 휴장일은 전일 값으로 보간
- 상장 전 데이터는 제거

평가

- 정규화된 오차율(NMAE)로 평가→종목별 가격 차이가 큰 경우에도 공정한 평가 가능

유전체 정보 품종 분류 AI 경진대회

대회 개요 & 문제 설정

- **대회명:** 유전체 데이터 분류 AI 경진대회 (Dacon 236035)
- **데이터:** 1만여 개의 SNP(유전자형) 정보 → **유전자형(class)** 예측
- **문제 특성:**
 - 고차원 **범주형 SNP 데이터**
 - 클래스 **불균형 심함**
- **목표:**
→ 범주형 구조와 불균형을 모두 고려한 **정확도 높은 분류 모델 개발**



유전체 정보 품종 분류 AI 경진대회

데이터 분석 & 피처 엔지니어링

- 문자열 결합 기반 피처 생성
 - 여러 SNP를 묶어 “염색체 단위” 결합
- 염기 카운트 특징
 - numGC, numA, numGC², sub, $H=3*\text{numGC}+2*\text{numA}$
→ 유전자의 염기 조성 패턴 반영
- 범주형과 수치형 분리 후 스케일링
 - 범주형은 인코딩, 수치형만 StandardScaler 적용

유전체 정보 품종 분류 AI 경진대회

인코딩 + 샘플링 + 모델링 전략

✓ CatBoostEncoder (Multiclass 확장)

- 각 범주형 변수와 class 간의 관계를 수치화
- 클래스별 이진 타깃 분해 -> 인코딩 -> 결합
- > 멀티클래스에서도 효과적인 타깃 인코딩 달성

✓ BorderlineSMOTE

- 결정 경계 근처 샘플만 증강해 불균형 완화 + 과적합 방지

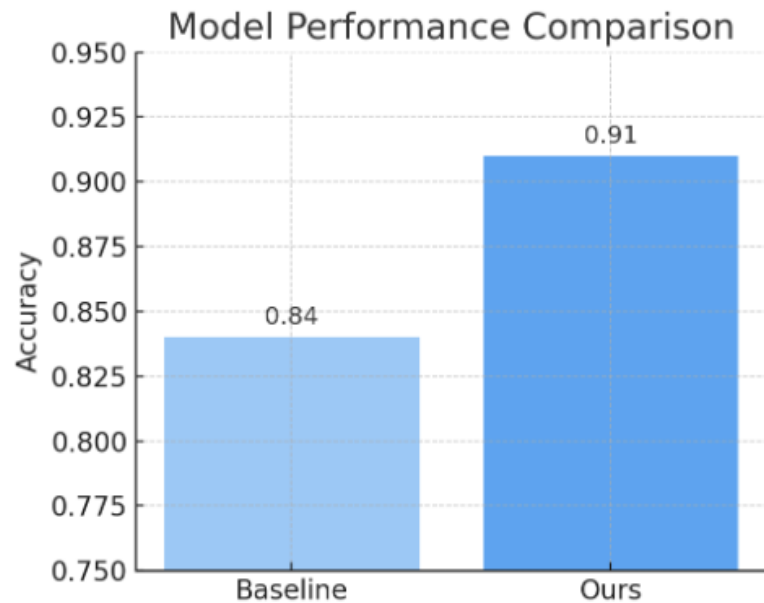
✓ VotingClassifier (Hard Voting)

- 다양한 모델 결합: XGB, RF, SVC, Ridge, GB, Bagging 등
- 가중치: XGB 2, RF 2 → 리더 모델 영향 강화

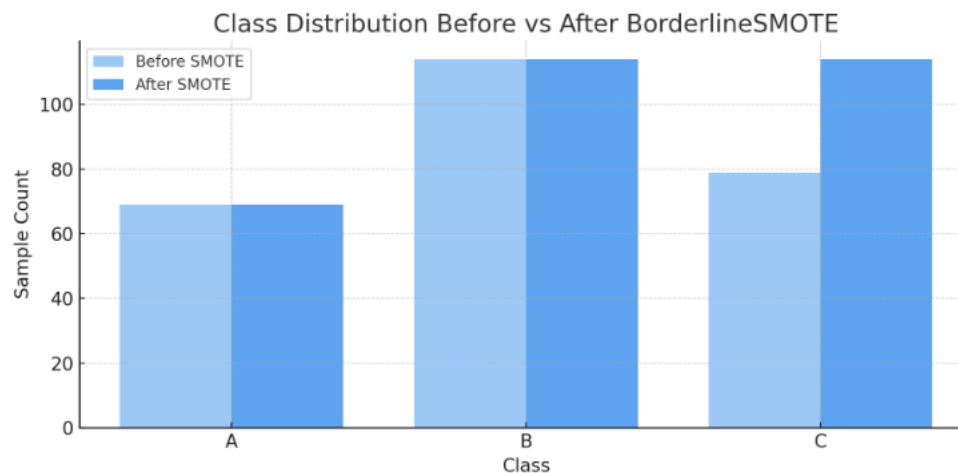
유전체 정보 품종 분류 AI 경진대회

결과 & 성능

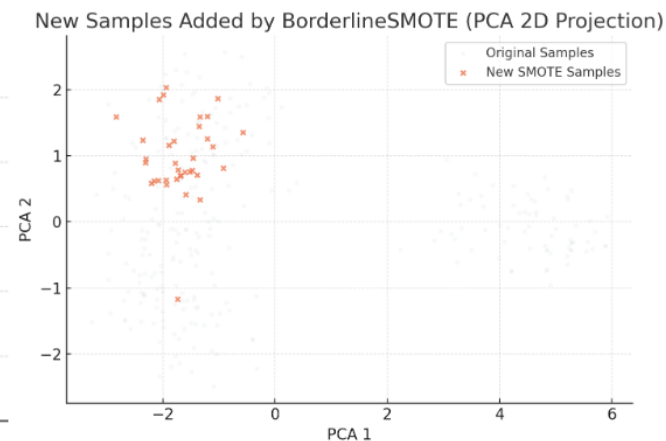
Baseline vs Ours 성능 비교 막대그래프



클래스 불균형 해소



새로운 샘플 생성



유전체 정보 품종 분류 AI 경진대회

배운 점 & 한계

- 배운 점:

- 도메인 이해가 모델 선택보다 더 중요
- 인코딩 + 샘플링 + 앙상블 = 일관된 파이프라인 설계의 힘

- 개선 아이디어:

- StratifiedKFold CV로 검증 안정화
- Optuna로 XGB/RF 하이퍼튜닝
- Soft Voting or SHAP 분석으로 해석력 강화



감사합니다